Field Hours

CSCE 608 Database Systems

Original notes for this site. Same topics as the catalog course. Not a reprint of another university's lectures, and not academic credit.

Lecture 1 · 45 minutes

Relations and a query

A relation is a set of tuples over a fixed set of attributes. Sets do not have duplicate rows and do not have an order. SQL bags and SQL's unordered results are the practical version, and the difference matters when you count. A key is a set of attributes that identifies a tuple. A foreign key is an attribute whose values are required to be keys in another relation. The requirement is a constraint, checked by the system, not a comment in a diagram.

Relational algebra is the theory under the query language. Selection keeps rows. Projection keeps columns. Join matches rows on a predicate, usually equality of a key and a foreign key. You can write the same question as a join or as a nested lookup. The algebra lets you rewrite one into the other and know the answer does not change. Query optimization is those rewrites plus a cost.

Use a schema small enough to compute by hand. Clerks have an id and a name. Jobs have an id, a clerk id, and a title. Pay has a job id and an hourly rate. The question 'what does this clerk earn on each job' is a join of all three, then a projection. If a clerk has no job, an inner join drops them. An outer join keeps them with an empty job. Pick the one the question asked for.

Normalization is what you do when a fact is stored twice and the two copies can disagree. If the rate lives on both the job and a separate pay table, an update can change one. Third normal form is the usual target: every non-key fact depends on the key, the whole key, and nothing but the key. A table that violates it is not illegal. It is a bug you have scheduled.

Three relations. The arrows are foreign keys, not a drawing of the disks.

Pay.job_id points at jobs

Jobs.clerk_id points at clerks

Clerks.id is the key

One join, computed by hand

1. Clerks: (1, Ana), (2, Bo). Jobs: (10, 1, 'doors'), (11, 1, 'roof'). Pay: (10, 20), (11, 25).

2. Join jobs to pay on job id, then to clerks on clerk id. Two result rows, both for Ana.

3. Bo disappears in an inner join. He has no job. If the question was 'list every clerk', this query is the wrong query.

Problem 1. Add a second pay row for job 10 with rate 30, and no unique constraint. What does the join return for that job?

Answer. Two rows for job 10, rates 20 and 30, both joined to Ana and doors. The bag of answers doubled because the fact was stored twice and the schema allowed it. A primary key on pay.job_id

would have refused the second row.

Problem 2. Why is 'the result came back sorted by name' not a property you may code against?

Answer. A relation has no order. The engine sorted because it was convenient, often because of an index, and it can stop. If you need an order, the query must say so.

Lecture 2 · 40 minutes

Pages and buffers

The disk and the memory do not trade rows. They trade pages, fixed-size blocks, often a few kilobytes. A heap file is a pile of pages of tuples. A slotted page puts a header at the front or a slot directory at the end, and the slots point at tuples inside the page. Variable-length tuples move around inside the page. The slot stays put, so an outside identifier, page id plus slot number, does not dangle when you compact the free space.

A buffer pool is a cache of pages, and it is the same idea as the architecture lecture with a different eviction cost. A hit avoids a disk read. A miss evicts some page. If the page is dirty, it must be written first. Pinning a page means a thread is using it and the eviction code must skip it. Forgetting a pin leaks a page that can be reused under your feet. Forgetting an unpin sticks the page in memory forever.

Replacement can be least recently used, or clock, as in an operating system. A database has a nasty sequential scan: it touches every page once, and a pure least-recently-used policy will evict the page you will need again in the inner loop of a join, in favor of a page you will never need. A policy that knows the scan is sequential can avoid caching those pages. The buffer manager is part of the query engine, not a generic cache you bolt on.

Physical layout decides the cost model. A table scan reads every page. An index, next lecture, reads a few. Until the index exists, every query plan you write is a scan plus a filter, and the cost is the number of pages, not the number of rows that pass the filter. A selective predicate on an unindexed column still reads the file.

A slotted page. Outside the page, the address is a page id and a slot, not a byte offset.

Slot directory

Free space

Tuple bytes

Page header

A pin leak

1. A scan pins page 1, reads it, and returns without unpinning.

2. The pool is full of pinned pages. The next miss has no victim.

3. The statement waits or fails. The data are fine. The bug is the missing unpin on the error path as well as the success path.

Problem 1. A tuple grows and moves inside the page. Why does page id plus slot still find it?

Answer. The slot directory is updated to the tuple's new offset. Callers hold the slot number, not the offset. A caller that cached the byte offset is now wrong, and that is why the interface does not hand

out byte offsets.

Problem 2. A join's inner side is a one-page table reused for every outer row. The buffer uses least recently used, and a big scan shares the pool. What goes wrong?

Answer. The scan's pages look newer than the one-page table, so the inner page is evicted and read back on every outer row. The pool should pin that inner page for the life of the join, or refuse to cache the scan.

Lecture 3 · 45 minutes

B+ trees

A B+ tree is a balanced search tree whose leaves hold the keys in order, usually with the row identifiers next to them. Internal nodes hold separator keys and child pointers. Every leaf is at the same depth, so a lookup is a short root-to-leaf walk, logarithmic in the number of keys for a large branching factor. The branching factor is what makes the tree short. A binary tree of a million keys is about twenty deep and pointer-chases through memory. A B+ tree of a few hundred keys per node is three or four deep and matches a page to a node.

Insert walks to the leaf. If the leaf has room, the key goes in order. If not, the leaf splits, the smallest key of the right sibling is copied up as a separator, and the parent gains a child. A parent that is full splits the same way. The root split is the only split that makes the tree taller. That is why the tree stays balanced without a rotation lecture.

Delete is the reverse and is easier to get wrong. A leaf that falls under a minimum occupancy merges with a sibling or borrows a key. Forgetting the parent separator leaves a key in the index that is not in the leaves, and the next range scan lies. Range scans are the reason for the leaf chain: once you are at the first key, you follow sibling pointers rather than walking the tree again.

A clustered index stores the rows in leaf order. A range becomes a sequential read. A secondary index stores identifiers, and each match may be a random page. The optimizer's choice between a scan and an index is this cost, not a belief that indexes are faster. An index on a column you never select is a write cost on every insert, because the tree must be updated inside the same transaction as the row.

A leaf split. The separator is copied up. The keys themselves stay in the leaves.

1. Leaf is full
2. Split into two leaves
3. Copy the middle key up
4. Parent points at both

Leaves that hold two keys

1. Insert 10, then 20. They share a leaf: 10, 20.
2. Insert 30. The leaf splits. Left leaf holds 10. Right leaf holds 20, 30. The parent stores separator 20 and points at both.
3. A search for 30 starts at the parent, sees 30 is not less than 20, and walks to the right leaf.

Problem 1. After that split, which keys live in the parent, and which live only in leaves?

Answer. The parent holds the separator 20 and two child pointers. The keys 10, 20, and 30 all live in leaves. 20 is copied up, not moved. A B+ tree search always ends at a leaf.

Problem 2. You range-scan from 10 to 30 and the right leaf was not linked to the left. What do you miss, and where should the link be?

Answer. You can find 10 by a tree walk and then have no way to the leaf that holds 20 and 30 without another search. Leaves must point at their right sibling. The parent is not a substitute for that chain.

Lecture 4 · 45 minutes

Joins and their cost

A nested-loop join reads each outer row and finds matches in the inner table. With no index that is a scan of the inner table per outer row. One thousand outer rows and five thousand inner rows is five million comparisons, and more importantly it is one thousand scans of the inner file. An index on the inner join key turns each outer row into a few page reads. The plan changed because the access path changed, not because the join condition changed.

A hash join builds a table on the smaller input and probes it with the larger. The cost is about one pass over each input, plus memory. If the hash table does not fit in memory, the join partitions both inputs and recurses. A graceful partition is a bit more than one pass. A hash table that thrashes the buffer pool is a nested loop in disguise. Know which one you wrote.

A sort-merge join sorts both inputs on the join key and walks them like a merge. It is attractive when the inputs are already ordered by an index, because then the sort is free. It also produces ordered output, which a later order-by or a merge join above it can use. The optimizer's job is to add these costs, including the output order as a credit, and pick the cheapest tree of operators.

Estimates come from statistics: how many rows, how many distinct keys, how the values spread. A stale statistic produces a confident wrong plan. A thousand-row estimate that is actually a million rows will pick a nested loop and then run for an hour. The lecture's demand is that you can compute the cost of two plans on the same numbers and see which assumption did the work.

Same join, two plans. The winner depends on memory and on whether an index exists.

Nested loop

Outer row, then inner

Fine with an index

Scans multiply without one

Hash join

Build, then probe

About one pass each

Spills if memory is short

One thousand outer rows, five thousand inner rows, no index

1. Nested loop compares about 1000 times 5000 = 5 million pairs, and it rereads the inner file.

2. Hash join builds on the smaller side, 1000 keys, and probes 5000 times. Comparisons are on the order of 6000 hash operations if nothing spills.

3. If the 1000-key table does not fit in the buffer pool, the hash join's cost jumps toward a partitioned join or a thrashing loop. The formula changed because memory changed.

Problem 1. The inner table has an index on the join key, and each lookup is 3 page reads. Outer table is 1000 rows. Roughly how many index page reads?

Answer. About 3000, plus the pages of the outer scan. Far under a repeated full scan of a large inner file. This is why the index exists.

Problem 2. Both inputs are already sorted on the join key. Which algorithm gets a free pass, and why?

Answer. Sort-merge. The sort steps cost nothing because the order is already there. You still pay the merge walk. A hash join would ignore the order and rebuild a table.

Lecture 5 · 45 minutes

Transactions, recovery, and two machines

A transaction is a sequence of reads and writes that should appear indivisible. The properties people recite are atomicity, consistency, isolation, and durability. Atomicity is the one recovery must provide: commit everything or keep nothing. Isolation is the one locking must provide: concurrent transactions should not observe each other's half-finished work. A system can offer a weaker isolation on purpose. Call it by its name. Do not call a dirty read serializable.

Two-phase locking acquires all the locks a transaction needs in a growing phase and releases them only in a shrinking phase. Strict two-phase locking holds the write locks until commit, which makes recovery simpler because you do not undo a write that another transaction already read. A deadlock is possible, as in the operating system lecture, and the database will abort one transaction. An abort is a legal outcome. The application must be able to retry.

Recovery uses a log. Each update is recorded before the data page is allowed to reach disk, so a crash can redo a committed change whose page never landed, and undo an uncommitted change whose page did. The log is the truth. A checkpoint limits how far you walk backward. This is the idea of write-ahead logging without a worksheet copied from a textbook: the log record hits stable storage first.

Two machines add a new failure. Two-phase commit asks each machine to prepare, then to commit. If the coordinator dies after every machine has prepared and before anyone hears the decision, the machines cannot commit and cannot abort. They are in doubt. Distributed data has failure modes a single log does not. A design that says the network is reliable has not done this lecture. The application-level retry rule from the networks course is the same bug seen from the other side.

Write-ahead log. The data page is not allowed to get ahead of the record that describes it.

1. Change the page in memory

2. Append a log record

3. Force the log

4. Only then write the page

A crash after a commit

1. The transaction commits. The commit record is on the log. The data page is dirty in memory and not on disk.

2. The machine loses power. Memory is gone. The log remains.

3. Recovery redoes the committed update onto the page. Without the log record, the commit would have been a lie.

Problem 1. Transaction A writes a row and releases the lock before it commits. Transaction B reads the row, and then A aborts. What did B read?

Answer. A value that does not exist. That is a dirty read. Strict locking holds A's write lock until A commits or aborts, so B cannot read the row in time to see it.

Problem 2. Both participants in two-phase commit have answered 'prepared', and the coordinator's decision message is lost. Why can they not simply abort?

Answer. The coordinator may already have told the other participant to commit, or it may have decided to commit and the log says so. Aborting a prepared transaction that the other machine committed splits the outcome. They must wait to learn the decision. That wait is the cost of the protocol.

Lecture 6 · 180 minutes

Relational algebra and SQL

The earlier lecture joined two tables by hand. This week is the language. Selection keeps rows. Projection keeps columns. A join combines rows that agree on a predicate. Union, difference, and renaming complete the algebra you need. SQL is a syntax for that algebra plus grouping and aggregation, which the pure algebra did not need for the first lecture.

The order of operations changes the cost and not the answer, if you rewrite legally. Filtering before a join is legal when the filter mentions only one side. Projecting away a column you later join on is not legal. An optimizer searches the legal rewrites. You should be able to spot an illegal one.

NULL is not a value. A comparison with NULL is unknown, and unknown fails a WHERE clause. That surprises people who expected false. An outer join exists to keep a row that matched nothing, and the missing side is NULL. If you then filter on the missing side, you may delete the rows the outer join added.

Grouping computes an aggregate per group. Every column in the output is either grouped or aggregated. A column that is neither is a query the language should reject. If a system accepts it, you do not know which row's value you got.

A legal rewrite.

Start from the joins -> Push a one-sided filter down -> Then project -> Do not drop a join key early

One join, filtered

1. Students have ids and majors. Enrollments have student ids and course ids. You want computer-science students in course 629.

2. Filter students to the major first, then join. The result matches filtering after the join, and the join is smaller.

3. Project students down to the major alone before the join and you can no longer join. The rewrite is illegal. The optimizer that does it is wrong, and so is the person who deleted the column in a subquery.

Problem 1. A WHERE clause says name = NULL. How many rows match, if some names are NULL?

Answer. None. The comparison is unknown, not true. Use IS NULL. Equality is the wrong predicate for NULL.

Problem 2. An outer join keeps students with no enrollment, and a later WHERE says course id = 629. What happens to the unenrolled students?

Answer. They disappear. Their course id is NULL, and NULL = 629 is not true. Put the course filter in the join condition if you meant to keep the unenrolled rows, or accept that the WHERE cancelled the outer join.

Lecture 7 · 180 minutes

Dependencies and normal forms

A functional dependency X -> Y means that two rows that agree on X agree on Y. It is a constraint on the legal tables, not a property of the current contents. A table that happens to satisfy it today and can violate it tomorrow does not have the dependency.

A key determines every other column. Third normal form says every non-key column depends on a key, a whole key, and not on another non-key. The definition used in practice is: for every dependency X -> A, either X is a superkey or A is part of a key. Boyce-Codd is stricter and sometimes splits tables in a way that loses a dependency you then have to enforce elsewhere.

A decomposition is lossless if the natural join of the pieces returns the original table, no more and no less. It is dependency-preserving if every dependency can be checked inside one piece. You can have one without the other. A lossy decomposition returns extra rows and silently invents facts.

Normalize a table that mixes two entities. Do not normalize a table whose remaining dependencies you did not write down. The normal form is a statement about dependencies you claimed, not about the shape of the spreadsheet.

One table or two.

Mixed

Student, major, advisor's phone

Phone depends on advisor, not on student

Decomposed

Student and advisor

Advisor and phone

The phone is stored once

A phone number that changes

1. Rows: (Ada, systems, 555-0100), (Ben, systems, 555-0100), both advised by Kim. The dependency advisor -> phone holds if Kim has one phone.

2. Update Ada's row to 555-0199 and Ben still shows 555-0100. The dependency is violated. The table allowed two phones for one advisor.

3. Split into enrollment-or-student rows that store the advisor, and an advisor table that stores the phone. The update touches one row. The join gets the phone back. If the split drops the advisor from the student side, the join cannot reconstruct the original, and the decomposition was lossy.

Problem 1. You decompose a table and the join has more rows than you started with. What did you lose?

Answer. The lossless condition. The pieces no longer determine which combinations existed. Extra rows are invented facts. Put back a key that ties the pieces together, or do not decompose that way.

Problem 2. A table is in third normal form for the dependencies you wrote, and the application still stores two birth dates for one person. Where is the bug?

Answer. In the missing dependency person -> birth date, or in a missing key. Normal form does not repair a constraint you never declared. Declare it and enforce it.

Lecture 8 · 180 minutes

Choosing a plan

A query has many legal plans. A cost-based optimizer estimates the cost of each and picks the cheapest estimate. The estimate is built from cardinalities and from the cost of scans, index lookups, and joins. A wrong cardinality produces a wrong plan that is still legal. Correctness and cost are separate.

Selections that remove most rows should happen early. A join order that starts from the selective input builds smaller intermediate results. The System R approach considers left-deep trees and keeps, for each subset of tables, the cheapest plan that produces it. You do not need to implement that search to see why an index on the selective predicate changes the winner.

Statistics are how the optimizer knows a predicate is selective. A stale histogram that thinks a column has two values, when it has two million, will pick a scan or an index for the wrong reason. Analyze the table after a bulk load. The plan can change under you, and that is the optimizer doing what you asked.

Look at the plan when a query is slow. A sequential scan of a table you thought was indexed means the index does not match the predicate, or the optimizer thinks the index is not worth it. Read the predicate. A function wrapped around the column defeats an ordinary index.

From a query to a plan.

1. Legal rewrites

2. Estimate cardinalities

3. Cost the candidate trees

4. Pick the cheapest estimate

One thousand outer rows

1. Outer table 1000 rows, inner table 5000, and a predicate on the outer table matches 10 rows. Nested loops without an index is 1000 times 5000 probes if you join first.

2. Filter the outer table first: 10 times 5000, or 10 index lookups if the inner join key is indexed. The earlier lecture's unindexed nested loop was the expensive plan.

3. If the statistic says the predicate matches 900 rows, the optimizer may scan. Update the statistic and the plan can flip. The query text did not change.

Problem 1. An index exists on name and the predicate is lower(name) = 'ada'. Why might the plan ignore the index?

Answer. The index is on name, not on lower(name). Every row's function value is a different expression. A scan, or an index on the expression, is what the predicate allows. The index you have does not answer it.

Problem 2. The optimizer's estimated rows are 10 and the actual rows are 100000. The plan was legal and slow. What do you fix first?

Answer. The statistics or the estimate, not the query's meaning. The plan was chosen for a different cardinality. Analyze the table, or add a better histogram, and recheck the plan before you rewrite the SQL.

Lecture 9 · 180 minutes

Serializability

Transactions are the unit of atomicity and of isolation. A schedule is serializable if its effect on the data is the effect of some serial order of the committed transactions. Two people reading a balance, each adding a deposit in their head, and each writing back, can lose one deposit. Both transactions committed. The schedule was not serializable.

Two-phase locking requires a transaction to acquire all the locks it will need before it releases any. The growing phase, then the shrinking phase. It ensures conflict serializability if you use shared locks for reads and exclusive locks for writes. Deadlock is possible. The earlier operating-systems course told you how a cycle forms. Here the locks are on records.

A weaker isolation level lets a transaction see another transaction's uncommitted or later-committed data. It runs faster and it admits anomalies you must name. Read committed does not prevent the lost update by itself if both transactions read before either writes, unless the write takes a lock that conflicts. Know which anomaly your level allows before you pick it for speed.

The lock manager is not the log. Locks are isolation. The log is durability and atomicity. A system can have one and not the other. You want both, and they are different mechanisms.

Two deposits.

Serializable

T1 reads 100 and locks

T2 waits

T1 writes 150 and unlocks

T2 reads 150 and writes 180

Lost update

Both read 100

T1 writes 150

T2 writes 130

The 50 deposit is gone

A balance of 100

1. T1 deposits 50. T2 deposits 30. The serial results are 180, in either order.

2. If both read 100 and both write their sum, the second write keeps 150 or 130 and drops the other deposit.

3. Exclusive locks on the read-for-update make the second transaction wait, reread, and add 30 to 150. Two-phase locking holds the lock until the transaction is ready to finish.

Problem 1. A transaction releases a lock on row A and then acquires a lock on row B. What two-phase locking forbids, and what bug does that allow?

Answer. It forbids acquiring after releasing. Another transaction can change A after the release and change B before the acquire, so this transaction's view of A and B never existed together. That is a failure of serializability.

Problem 2. You turn off locking and keep the write-ahead log. Which property did you keep?

Answer. Durability and the ability to undo a crash, if the log is complete. You gave away isolation. Concurrent transactions can still lose updates. The log does not order them.

Lecture 10 · 180 minutes

Recovery

Write-ahead logging means a change is on the log, on stable storage, before the data page is allowed to hold it. A crash then has a source of truth. Recovery redoes committed work that did not reach the pages and undoes uncommitted work that did. The earlier lecture's 'crash after commit' is the redo half.

A checkpoint bounds how far back recovery must read. Fuzzy checkpoints let transactions keep running and record a starting point. You do not need every optimization to state the invariant: after recovery, every committed transaction's effects are present and every uncommitted transaction's effects are absent.

Compensation records are how undo is itself logged, so a crash during recovery can be recovered. If undo changed a page and did not log the undo, a second crash has no idea the undo happened. The log is the only structure you trust after a crash. The pages are suspects.

A commit point is the moment the commit record is stable. A transaction that printed 'done' to a user before that record is durable has lied. The application waits for commit.

Recovery.

1. Read the log from the checkpoint

2. Redo winners

3. Undo losers

4. The commit record is the difference

A crash after a commit record

1. The log contains an update and a commit for T1. The data page was not flushed.

2. Recovery redoes the update. T1's effect is present. There is nothing to undo.

3. If the commit record had not reached stable storage, T1 is a loser. Recovery undoes the update if it reached the page. The user was not told success, if the application waited.

Problem 1. The data page is written before the log record, and the crash happens between them. Can recovery reconstruct the change?

Answer. No. The page holds a change the log has never seen, or a torn version of it, and the log cannot redo or undo what it does not contain. Write-ahead exists to forbid this order.

Problem 2. Recovery crashes halfway through undo. What lets the second recovery finish?

Answer. The undo steps were logged as compensation records. The second recovery sees them and does not undo the undo. Without those records the second pass can corrupt the page.

Lecture 11 · 180 minutes

Two machines

A transaction that must commit on two databases cannot decide alone. Two-phase commit elects a coordinator. Each participant votes after preparing, which means its local log can commit or abort as instructed. If every vote is yes, the coordinator writes a commit and tells them. If any vote is no, or a participant is silent past the timeout you defined, the coordinator aborts.

The window is the coordinator's commit record. A participant that voted yes and has not heard back must keep the transaction prepared and hold its locks. It cannot unilaterally commit, and it cannot unilaterally abort if the coordinator might have committed. That uncertainty is the cost of the protocol. A timeout alone does not resolve

it.

Replication of a single log is a different design. One leader orders the commits. Followers apply the log. Failover promotes a follower that has the log. Split brain, two leaders, is the failure you design against with a majority. Two-phase commit across independent authorities and replication inside one service answer different organizational problems.

The capstone's crash test can be this protocol in miniature: two logs, a coordinator, a killed process after the prepare and before the commit message. You should be able to say what each side is allowed to do next.

Two-phase commit.

Prepare -> Vote -> Coordinator decides -> Participants learn the decision

A vote that gets lost

1. Participant A votes yes and prepares. The coordinator decides to commit and crashes before A's message leaves.

2. A holds locks. It does not abort on a timer, because the coordinator may have told B to commit already.

3. When the coordinator recovers, its commit record tells it to resend the decision. A applies it. The uncertainty ends when the decision is known, not when A is impatient.

Problem 1. Both participants voted yes and the coordinator has no stable commit record, then the coordinator crashes. What is the decision on recovery?

Answer. Abort. Without a stable commit, the coordinator did not decide to commit. It tells the participants to abort. They are still waiting, prepared, and they can roll back.

Problem 2. Why is a timeout not enough for a prepared participant to abort?

Answer. The coordinator may already have committed and told the other participant. Aborting locally would make one side committed and one aborted. The participant must learn the decision. The protocol blocks rather than guess.

Lecture 12 · 180 minutes

Physical design

An index speeds the predicate it matches and slows the writes that must maintain it. A table with six indexes pays six updates on every insert. Build the index that the slow query uses. Do not build one per column as a ritual.

A clustered index stores the rows in key order. A range on that key is sequential I/O. A secondary index stores pointers and may turn a range into random fetches of the rows. The earlier B+ tree lecture is the structure. Clustering is the decision about which key matches the layout.

A covering index holds every column the query needs, so the plan need not touch the row. It is larger. It is justified when the query is frequent and the table is wide. Explain the query before you add the columns.

Denormalization copies a column to avoid a join. It reintroduces the update anomaly from the normal-form

lecture. Do it when the measurement says the join dominates and you have a way to keep the copy honest. Do it as a default and you will store two phones for one advisor again.

Clustered or not.

Clustered on the key

Rows sit in key order

A range is a scan of neighbors

Secondary index

The index holds pointers

A range becomes many fetches

Leaves that hold two keys, and a range

1. A B+ tree on course id, clustered, leaves hold (629, row) and (633, row) as neighbors if the ids are adjacent in the key order you chose.

2. A query for ids between 629 and 640 reads those leaves in order.

3. The same keys in a secondary index yield pointers. If the rows live on different pages, the range costs a random read per row. The tree height was never the expensive part.

Problem 1. A table is inserted into constantly and queried by primary key once a day. How many secondary indexes do you want?

Answer. None, until a query needs one. Each secondary index taxes the inserts every day for a lookup that does not happen. The primary access already has a structure.

Problem 2. You copy a customer's city onto every order to avoid a join, and the customer moves. What must the write path do?

Answer. Update every copied city, or accept that orders show different cities for one person. The copy is a dependency you reintroduced. The write path has to maintain it, or the denormalization is a bug.

Lecture 13 · 180 minutes

A schema for the capstone

The capstone stores records that more than one request may touch. Design the schema this week, before the code. Write the entities, the keys, and the dependencies. Put it in third normal form unless you write the dependency you are choosing to violate and the code that repairs it.

Name the transactions. A transfer of a record from one owner to another is one transaction, not two requests that each commit. The serializability lecture is why. Name the isolation you need. If two requests can lose an update, you need the locks, not a comment.

Name the crash. Which log record is the commit. What recovery must redo. If there are two stores, say whether you are using two-phase commit or a single log. The design is the sentence that answers that. The code comes after.

Estimate one query. What index it uses, what it would cost without the index, on a thousand rows and on a million. The number can be rough. It cannot be absent.

The design note.

Entities, keys, dependencies

Transactions and their isolation

The crash and the log

One query and its index

A record and an audit row

1. Updating a record and inserting an audit row must commit together. If only the audit row commits, the history describes a change that is not there.

2. One transaction, one commit record, covers both writes.

3. A crash after the commit redoes both. A crash before the commit undoes both, or never made them visible. That is the whole specification of this pair.

Problem 1. Two requests update the same record and each commits its own transaction after reading the old value. Which lecture names the bug?

Answer. The serializability lecture. It is a lost update. The schema being in normal form does not prevent it. The transactions need conflicting locks or a conditional update that fails if the value changed.

Problem 2. You store the user's display name on every audit row. What dependency did you create?

Answer. Audit row -> display name is false if the name lives on the user. User id -> display name is the dependency, and copying the name means an update must find every audit row or the copies will disagree. Write that down or store the id only.

Lecture 14 · 180 minutes

The database you can defend

You should be able to do five things on paper. Translate a small question into a join and say which filter pushes down. Name a dependency and decompose without losing rows. Read a plan and say why an index was ignored. Show a lost update and the lock that prevents it. Say what a commit record means after a crash.

The hours for this course are those five, worked without the notes, plus the schema note for the capstone. The opening lectures introduced the pieces. These weeks are the course.

If a tool picks a plan for you, you are still responsible for the plan. Print it. If a tool picks an isolation level, name the anomaly it allows. Defaults are not knowledge.

Nothing here is a manual for a particular product. The mechanisms are the ones every transactional store is judged against. Use a product in the capstone, and map its words onto these.

Five checks.

A legal rewrite

A lossless split

A plan and a statistic

A lock order

A commit record

Map a product word

1. A product says 'repeatable read.' You look up which anomalies it still allows. You do not assume the English words match the textbook level of the same name. Dialects differ.

2. You write down the anomaly your capstone cannot tolerate, and you confirm the level forbids it, with a test of two sessions.

3. The test is two transactions, the same one as the lost-update example. If the second write silently overwrites, the level is not the one you needed.

Problem 1. Recovery restores committed work and your application still shows the user a success it printed before commit returned. Where is the bug?

Answer. In the application. It claimed durability before the commit record was stable. Wait for commit, then tell the user. Recovery did what it could.

Problem 2. Which of the five checks fails if the join of your decomposed tables grows extra rows?

Answer. The lossless split. The other four can pass and the schema is still wrong. Extra rows are invented facts. Fix the decomposition before you index it.